

Empu Harga

A Calibrated Quantile Regression System for
Indonesian Used Motorcycle Price Suggestion
Theory, Architecture, Implementation, Results

Hilmi Nur Ardian

`master.ardian@gmail.com`

Repository: <https://github.com/masterA88/rekomendasi-harga-demo>

Live demo: <https://web-sepia-seven-27.vercel.app>

Document version: 2026-05-21

Abstract

Empu Harga is a price suggestion system for Indonesian used motorcycle listings. The user fills a seller form with brand, model, year, mileage, condition, and kecamatan (sub district), and the system returns three numbers, namely a lower bound, a recommended price, and an upper bound. The widget is advisory: the seller can override the suggestion, and the platform never binds the listing to it. The methodology combines LightGBM quantile regression at three target quantiles, Conformalized Quantile Regression for finite sample marginal coverage, and a spatially weighted recalibration step that restores conditional coverage across kecamatan of differing data density. A three layer hierarchical fallback ensures the predictor always returns a valid interval, even on rare brands and previously unseen sub districts. The serving architecture is intentionally minimal, namely a precomputed cell cache materialised to DuckDB, baked into per brand JSON files, and shipped from the Vercel content delivery network with zero ongoing cost. A Wasserstein 1 drift detector is provided as a production monitor that does not break when new categorical levels appear, in contrast to the legacy Population Stability Index. This document walks through the business problem, every theory the system relies on, the code modules that implement them, the offline and online data flow, the empirical results, and an annotated bibliography that connects every citation to the code or design decision it shaped.

Contents

1	Business Problem and Motivation	3
1.1	Why this is a hard problem	3
1.2	The product framing chosen	4
2	Glossary	4
2.1	Indonesian administrative and product vocabulary	4
2.2	Machine learning vocabulary	4
3	Theoretical Foundations	6
3.1	The problem class as conditional quantile interval regression	6
3.2	Why gradient boosted decision trees on tabular data	6
3.3	Quantile regression and the pinball loss	6
3.4	Conformalized Quantile Regression	7
3.5	Spatially weighted recalibration	7

3.6	Hierarchical target encoding with Bayesian smoothing	8
3.7	Temporal split and honest evaluation	8
3.8	Three layer hierarchical fallback for cold start	8
3.9	Wasserstein 1 distance for covariate drift detection	9
3.10	Hyperparameter search by tree structured Parzen estimator	9
3.11	Serving architecture, precompute and cache versus live inference	10
3.12	Mileage correction at serve time	10
3.13	Confidence inflation for cold start	11
3.14	Deal tier classification	11
3.15	The labels question, listed price as a noisy proxy for sold price	11
3.16	Feedback loop discipline in advisory price suggestion	12
3.17	Marketplace experimentation, switchback design and interference bias	12
4	Mathematical Summary	12
5	Dataset	14
6	End to End Data Flow	15
6.1	Offline pipeline	15
6.2	Online pipeline	16
7	Layer by Layer Architecture	16
7.1	Layer 1, Data Reference and Loading	16
7.2	Layer 2, Feature Engineering	16
7.3	Layer 3, Model Training	17
7.4	Layer 4, Calibration	17
7.5	Layer 5, Unified Inference	17
7.6	Layer 6, Stress Test	17
7.7	Layer 7, FastAPI Service	18
7.8	Layer 8, Cell Cache and Static Bake	18
7.9	Layer 9, Static Web Demo	18
7.10	Layer 10, Drift Monitoring	18
7.11	Layer 11, Cost Modeling	19
7.12	Layer 12, Preflight Verification	19
8	Story, Mistakes and Corrections	19
9	Results	20
10	Limitations	22
11	Potential Future Projects	23
12	Repository Layout	24
13	Disclaimer	26
A	Annotated Bibliography	27
A.1	Quantile regression and conformal calibration	27
A.2	Gradient boosting on tabular data	27
A.3	Hyperparameter search	28
A.4	Drift detection and monitoring	28
A.5	Marketplace pricing references	28
A.6	Industry case studies	29

A.7 Indonesian market and geography references	29
A.8 Tabular methods, additional	30
A.9 Software references	30

1. Business Problem and Motivation

Indonesia is the third largest motorcycle market in the world. The 2025 production volume reached roughly 6.4 million units, the total stock is estimated in the low tens of millions, and the used motorcycle segment is a significant fraction of household mobility expenditure in the Jabodetabek metropolitan region (Jakarta, Bogor, Depok, Tangerang, Bekasi). When an individual seller lists a used motorcycle on an online marketplace, the most fragile moment of the transaction is the moment the seller types a price into a form field. Too high and the listing sits idle for weeks. Too low and value is left on the table. Both outcomes hurt the marketplace, because both reduce liquidity and trust.

A modern marketplace addresses this with an in form *price suggestion widget*. As the seller fills the metadata of the listing, the widget displays a recommended price and a plausible range around it, scoped to the seller’s sub district. The Indonesian language label for this widget is *Rekomendasi Harga*, namely Price Recommendation. The widget is advisory, not binding. The seller can ignore the recommendation. The disclaimer next to it states verbatim, “*Harga bersifat estimasi sesuai kondisi dan harga pasaran. Kamu tetap bisa menentukan harga sesuai keinginanmu*”, namely, the price is an estimate consistent with condition and market prices, and the seller is free to choose any value.

1.1 Why this is a hard problem

The naive view treats the widget as a regression on a few simple features such as brand, year, and mileage. The honest view is that the problem class hides four distinct difficulties.

1. **The user wants an interval, not a point.** Three numbers are displayed (low, recommended, high). The recommended value is a conditional median. The low and high values are conditional quantiles at ten percent and ninety percent. Treating this as a single point regression and then post hoc widening with a fixed multiplier is a classic anti pattern that produces intervals that are systematically too tight in volatile cells and too loose in stable ones.
2. **Coverage must be honest.** If the widget promises that ninety percent of true sale prices fall inside the band, then approximately ninety percent really should. A well known empirical failure mode of vanilla quantile regression is to undercover, namely to display narrower bands than the data supports.
3. **Geography matters at a small administrative grain.** The Indonesian administrative hierarchy is provinsi, then kotamadya or kabupaten, then kecamatan, then kelurahan. A Honda Vario 125 with 60,000 kilometers in the upscale Kebayoran Baru kecamatan should suggest a different price band than the same motorcycle in a peri urban Bekasi kecamatan. With 70 kecamatan in Jabodetabek alone, the model must condition on a high cardinality categorical with non trivial spatial structure.
4. **Brand concentration is extreme.** Honda and Yamaha together hold close to ninety seven percent of Indonesian two wheeler share. The interesting cold start case is not the rare brand, since rare brands almost never appear, but the freshly launched Honda model that did not exist in the training window. This is a transfer learning problem inside the dominant brand, not an out of distribution problem at the brand level.

1.2 The product framing chosen

The system frames the widget as a **listing side, advisory, conditional quantile interval regression**. It is closer in product spirit to the host side smart pricing on lodging platforms than to dynamic surge pricing on transport platforms. The marketplace does not commit any capital based on the model output, and the cost of a single incorrect suggestion is bounded by reputational and seller behaviour effects, not by balance sheet exposure. This framing is important because it sets the design temperature, namely how aggressively the model should be tuned versus how conservatively the system should hedge.

Why this framing is the right one. Advisory price suggestion on a classifieds platform is structurally different from instant offer (iBuying) businesses, in which the platform itself buys the vehicle at the model's price. In the iBuying case, the model error is balance sheet exposure. The Zillow Offers iBuying business famously absorbed losses on the order of three hundred million United States dollars when its valuation model overestimated home prices during a falling market. The same model, when used in the advisory configuration, performs well within tolerable bounds. The structural lesson is, the cost of error scales with the capital exposure, not with the sophistication of the algorithm.

2. Glossary

2.1 Indonesian administrative and product vocabulary

Rekomendasi Harga

Indonesian for “price recommendation”. The product name of the widget.

Provinsi Province. There are roughly thirty four in Indonesia.

Kotamadya City level administrative unit. *Kabupaten* is the rural equivalent.

Kecamatan Sub district. The granularity at which the widget reports the locale tag.

Kelurahan Sub sub district. Finer than kecamatan. Not used at the model level.

Jabodetabek

Greater Jakarta metropolitan region consisting of Jakarta, Bogor, Depok, Tangerang, Bekasi.

BPS *Badan Pusat Statistik*, the Indonesian central statistics agency.

Kondisi Condition tag of the motorcycle. Three values, namely mulus (pristine), normal, perlu perbaikan (needs repair).

Mileage km Total odometer reading in kilometers.

MSRP Manufacturer Suggested Retail Price, namely the new motorcycle price at year of manufacture.

2.2 Machine learning vocabulary

GBDT Gradient Boosted Decision Trees. The dominant family for tabular supervised learning.

LightGBM A leaf wise GBDT implementation by Microsoft, optimised for speed and memory.

Quantile regression

Regression on conditional quantiles of the response rather than the conditional mean. Trained with the pinball loss.

Pinball loss Asymmetric loss function whose minimiser is the conditional quantile at level α .

CQR Conformalized Quantile Regression (Romano, Patterson and Candès, 2019). Wraps any quantile regressor to deliver finite sample marginal coverage under exchangeability.

SW CP Spatially Weighted Conformal Prediction (Hjort, Hermansen, Pensar and Williams, 2025). Reweights calibration scores by spatial proximity so that conditional coverage holds per region.

Target encoding

Replacing a high cardinality categorical with the mean of the target conditional on the category. Bayesian smoothing pulls low support categories toward a parent mean.

Temporal split

Train, calibration and test partitions sorted by listing creation timestamp so that the evaluation respects the order in which data arrives in production.

Coverage Empirical fraction of true responses inside the predicted interval.

Pinball at quantile α

Value of the pinball loss when training a quantile regressor at level α . Lower is better.

MAPE Mean Absolute Percentage Error. Used for the point estimate (the median).

Wasserstein 1

First Wasserstein distance between two univariate empirical distributions. Used as a covariate drift detector.

PSI Population Stability Index. Legacy benchmark drift score. Breaks when a categorical level appears or disappears.

Total variation

Half the L1 distance between two empirical probability mass functions on a finite alphabet. Used as the categorical analogue of Wasserstein 1.

Optuna A hyperparameter optimisation framework using Tree structured Parzen Estimator sampling.

MLflow Open source experiment tracking, model registry and artifact store.

DuckDB An embedded analytical database that runs in process and reads Parquet natively.

FastAPI An asynchronous Python web framework with first class Pydantic schema validation.

Vercel A content delivery and serverless deployment platform. The hobby tier of Vercel is the substrate of the live demo.

Next.js A React based application framework with App Router. The static demo is built with Next.js sixteen.

3. Theoretical Foundations

This is the core section of the document. The system did not arise from intuition alone. Every architectural decision corresponds to a published result or a well established empirical practice. The eleven subsections below list, in build order, the theories the system relies on, and for each, explain how the theory is operationalised in code. The annotated bibliography in [Appendix A](#) connects every citation to the precise piece of the system it informed.

3.1 The problem class as conditional quantile interval regression

Let Y denote the listed price of a used motorcycle in Indonesian Rupiah, and let $X = (X_{\text{brand}}, X_{\text{model}}, X_{\text{year}}, X_{\text{mileage}}, X_{\text{kondisi}}, X_{\text{kec}})$ denote the feature vector. The system models the conditional distribution $Y | X$ and reports three conditional quantiles, namely the ten percent quantile, the median, and the ninety percent quantile,

$$q_{\alpha}(x) = \inf\{y \in \mathbb{R} : \Pr(Y \leq y | X = x) \geq \alpha\}, \quad \alpha \in \{0.1, 0.5, 0.9\}. \quad (1)$$

The recommended price is $q_{0.5}(x)$. The interval $[q_{0.1}(x), q_{0.9}(x)]$ is the “range bar” displayed under the recommendation. The target marginal coverage is eighty percent, so $\alpha = 1 - 2 \cdot 0.1 = 0.8$ where the coverage level is twice the lower miscoverage.

3.2 Why gradient boosted decision trees on tabular data

The empirical benchmark by Grinsztajn, Oyallon and Varoquaux (2022, NeurIPS Datasets and Benchmarks Track), titled *Why do tree based models still outperform deep learning on tabular data?*, evaluates a wide range of models across forty five medium scale tabular benchmarks (between ten thousand and one million rows, mixed numerical and high cardinality categorical features). The result is unambiguous, namely tree based ensembles dominate transformer and deep tabular baselines at this scale at a fraction of the compute. The closest specific paper for used vehicle pricing is Madhusudhanan, Hartwig, Loni and Schmidt-Thieme (2024), *ProbSAINT: Probabilistic Tabular Regression for Used Car Pricing*, which lands a deep tabular model that wins on probabilistic calibration metrics but requires a much larger operational footprint. The system therefore commits to LightGBM as the v1 backbone and treats ProbSAINT as a v1.5 ablation candidate for an optional research study.

Practical implication. For the operating point of the Indonesian used motorcycle market (a few tens of thousands of listings per month per metropolitan region), LightGBM trains on a laptop in minutes and serves predictions in microseconds on a single central processing unit core. Defaulting to a graphics processing unit endpoint for this load class is the single most common cost mistake in machine learning shops.

3.3 Quantile regression and the pinball loss

The conditional quantile at level $\alpha \in (0, 1)$ is the minimiser of the expected *pinball loss* (Koenker and Bassett, 1978). For a single observation (x, y) and a candidate prediction $\hat{y}$,

$$L_{\alpha}(y, \hat{y}) = \max(\alpha(y - \hat{y}), (\alpha - 1)(y - \hat{y})). \quad (2)$$

Three quantile regressors are trained independently at $\alpha \in \{0.1, 0.5, 0.9\}$ by passing `objective="quantile"` and `alpha= $\alpha$` to LightGBM. The three regressors share the same feature set and the same hyperparameters, namely the hyperparameters that win the Optuna search at the median ($\alpha = 0.5$).

Why fit three independent regressors instead of a single distributional model. The alternative is to fit a parametric distribution (LogNormal is the canonical choice for prices) and read quantiles off the fitted distribution. This approach is rejected for two reasons. First, the parametric family is misspecified at the tails. Vehicle prices have heavier upper tails than LogNormal because of rare high end variants. Second, three independent learners decouple the median accuracy from the interval calibration step, which is performed separately and downstream (Section 3.4).

3.4 Conformalized Quantile Regression

A common empirical failure of vanilla quantile regression is undercoverage, that is, the interval $[q_{0.1}, q_{0.9}]$ on held out data covers less than eighty percent of true responses. *Conformalized Quantile Regression* (Romano, Patterson and Candès, NeurIPS 2019) provides a finite sample marginal coverage guarantee under the mild assumption that calibration and test data are exchangeable.

Construction. Hold out a calibration set $\{(X_i, Y_i)\}_{i=1}^n$ from the training distribution. For each calibration point compute the *conformity score*

$$E_i = \max(q_{\alpha_{lo}}(X_i) - Y_i, Y_i - q_{\alpha_{hi}}(X_i)), \quad (3)$$

where $\alpha_{lo} = 0.1$ and $\alpha_{hi} = 0.9$ are the lower and upper target quantiles. Sort the scores in increasing order. The conformal quantile correction Q is the order statistic at rank

$$r = \lceil (n+1)(1-\alpha) \rceil, \quad \alpha = 0.2, \quad (4)$$

so that Q is the conformity score at position r . The conformalised interval is then

$$[q_{0.1}(x) - Q, q_{0.9}(x) + Q], \quad (5)$$

and Theorem 1 of Romano et al. states that under exchangeability,

$$\Pr(Y \in [q_{0.1}(X) - Q, q_{0.9}(X) + Q]) \geq 1 - \alpha. \quad (6)$$

The system uses the symmetric variant of CQR, namely the same Q widens both endpoints, because asymmetric variants buy little in exchange for substantial code complexity at the production layer.

3.5 Spatially weighted recalibration

CQR delivers *marginal* coverage. In production, marginal coverage hides the fact that the model overcovers in data dense kecamatan and undercovers in data sparse ones. The fix is *Spatially Weighted Conformal Prediction* (Hjort, Hermansen, Pensar and Williams, 2025), originally formulated for real estate automated valuation. The idea is to compute, for each test point, a per region conformal correction $Q(x)$ obtained as a weighted quantile of the calibration scores, where the weight $w_i(x)$ favours calibration points spatially close to the test point.

Hierarchical kernel. Indonesian administrative geography is categorical with a clean hierarchy (provinsi, kotamadya, kecamatan). The system encodes spatial proximity as a stepwise function on this hierarchy, namely

$$w_i(x) = \begin{cases} 1.00 & \text{if } X_{\text{kec}}^{(i)} = X_{\text{kec}}(x), \\ 0.40 & \text{if same kotamadya, different kecamatan,} \\ 0.10 & \text{if same provinsi, different kotamadya,} \\ 0.02 & \text{otherwise.} \end{cases} \quad (7)$$

The per region correction is the weighted $(1 - \alpha)$ quantile of $\{E_i\}$, that is

$$Q(x) = \min \left\{ t : \frac{\sum_i w_i(x) \mathbf{1}[E_i \leq t]}{\sum_i w_i(x)} \geq 1 - \alpha \right\}. \quad (8)$$

A safety fallback applies when the weighted effective sample size $\sum_i w_i(x)$ drops below thirty. In that case the system returns the marginal CQR correction Q from Section 3.4 instead of a degenerate weighted quantile. The threshold of thirty matches the Bayesian smoothing strength used in the target encoder (Section 3.6), which is a deliberate consistency between two independent calibration layers.

Why this layer is the most distinguishing one. Vanilla CQR is now standard practice. Spatially weighted recalibration is the addition that converts an interval system from “looks plausible on average” to “defensibly calibrated everywhere”. A reviewer asking “what is your coverage in low volume kecamatan?” should receive a table showing coverage staying inside a narrow band across data density deciles. The system actually delivers this, as documented in Section 9.

3.6 Hierarchical target encoding with Bayesian smoothing

LightGBM handles categorical features natively, but the categorical with the highest cardinality, namely kecamatan with seventy distinct values in Jabodetabek, benefits from an additional engineered representation. The system encodes kecamatan as a target mean smoothed toward the kotamadya parent mean.

Bayesian smoothing. Let $\bar{y}_k$ denote the empirical mean of Y in kecamatan k on the training set, n_k the number of observations in k , $\bar{y}_{p(k)}$ the empirical mean of the parent kotamadya $p(k)$, and m the smoothing strength. The encoded value is

$$\text{enc}(k) = \frac{n_k \bar{y}_k + m \bar{y}_{p(k)}}{n_k + m}, \quad m = 30. \quad (9)$$

For unseen kecamatan the encoder returns the parent kotamadya mean directly. The system chains the same logic at a higher level, namely a kotamadya is itself encoded as a target mean smoothed toward the provinsi mean. The full hierarchy is kecamatan toward kotamadya toward provinsi toward the global mean. The same encoder class is reused for additional high cardinality composite keys, specifically brand colon family, brand colon displacement band, and model colon year.

3.7 Temporal split and honest evaluation

The evaluation matters as much as the training. The system splits the dataset chronologically by listing creation timestamp into seventy percent training, fifteen percent calibration, fifteen percent test. The temporal split mimics the deployed regime where the model is updated nightly and scores listings that did not exist at training time. Random splits inflate the apparent accuracy because they leak future market conditions into the training set, which would not happen in production.

3.8 Three layer hierarchical fallback for cold start

The system is designed so that a request never fails, even on edge cases that the training set did not see. The fallback is layered.

1. **Feature layer (target encoder).** An unseen kecamatan returns the kotamadya mean, then the provinsi mean, then the global mean. The encoder never returns a missing value.

2. **Calibration layer (SW CP).** An unseen region with weighted effective sample size below thirty falls back to the marginal CQR correction. The conformal interval remains valid.
3. **Confidence layer (predictor).** The serving layer inflates the half width of the interval when the request hits an unseen brand model pair (multiplier of one point three zero) or an unseen kecamatan (multiplier of one point one five), and surfaces a `confidence_flag` string. The frontend renders the flag as “estimasi luas” (wide estimate) so the user understands the wider band.

The three layers are independent. A request that misses every previous filter still gets an honest, wider interval rather than a refusal.

3.9 Wasserstein 1 distance for covariate drift detection

Once the system is deployed, the inventory distribution will drift over time. New motorcycle models launch, old ones are discontinued, fuel subsidy reform shifts the displacement mix, and macro events change the absolute price level. A drift detector is needed.

Why not Population Stability Index. The legacy approach is PSI, namely

$$\text{PSI}(P, Q) = \sum_b (p_b - q_b) \log \frac{p_b}{q_b}, \quad (10)$$

where the sum is over the bins b of the feature histogram. The fundamental problem is that PSI returns positive infinity when a bin has zero mass in either the reference window or the candidate window. The expected production failure mode is the launch of a new motorcycle model. As soon as the new model appears in inventory, PSI on the brand colon model feature evaluates to positive infinity for any number of days. The alert that is supposed to catch genuine drift instead saturates on every product launch.

Wasserstein 1 distance. The first Wasserstein distance between two univariate empirical distributions P and Q is the area between their cumulative distribution functions,

$$W_1(P, Q) = \int_{-\infty}^{+\infty} |F_P(x) - F_Q(x)| dx. \quad (11)$$

For two finite samples of size n and m the integral becomes a finite sum over the union of the support, computable in $\mathcal{O}((n+m) \log(n+m))$ time. The quantity is finite under support changes, namely it does not require shared support, and it reports in the units of the underlying feature. An alert at “ $W_1 = 380,000$ rupiah on median listed price week over week” is interpretable. An alert at “ $\text{PSI} = +\infty$ ” is not.

Categorical analogue. For categorical features the system uses the total variation distance,

$$\text{TV}(p, q) = \frac{1}{2} \sum_a |p_a - q_a|, \quad (12)$$

which is finite (bounded by one) whenever a new categorical level appears.

3.10 Hyperparameter search by tree structured Parzen estimator

The system tunes the median LightGBM with Optuna using the Tree structured Parzen Estimator (TPE) sampler of Bergstra, Bardenet, Bengio and Kégl (2011). TPE models two densities, $\ell(x)$ for hyperparameter configurations with objective below a quantile threshold and $g(x)$ for the rest, then samples new configurations that maximise the ratio $\ell(x)/g(x)$. The search space spans

nine hyperparameters, namely the number of estimators, the learning rate (logarithmic prior), the number of leaves, the minimum child samples, the feature fraction, the bagging fraction, the bagging frequency, and two regularisation strengths (logarithmic priors on L1 and L2). The lower and upper quantile regressors inherit the median’s best hyperparameters. The search runs for between twenty and fifty trials.

3.11 Serving architecture, precompute and cache versus live inference

The widget is on the seller listing form. Latency targets are tight (sub fifty milliseconds at the ninety nine percent percentile for the cached path). The natural decision is between two architectures.

Live inference. Feature extraction and three LightGBM predictions are executed per request. Pros, namely a single source of truth and no cell grid limitations. Cons, namely higher tail latency, higher compute cost, no straightforward content delivery network distribution.

Precompute and cache. The Cartesian product of $\{\text{brand, model}\} \times \text{year} \times \text{kecamatan} \times \text{kondisi}$ is finite. The system materialises every cell, namely $35 \times 11 \times 70 \times 3 = 80,850$ cells, into a DuckDB read only file with an index on the lookup keys. At serve time the request is an indexed lookup, the mileage adjustment is applied as a small linear correction on the cached cell value, and the response is returned in microseconds. The committed v1 architecture is precompute and cache, with live FastAPI inference as the fallback when the cell grid expands beyond what can be statically pre baked. The trade off accepts a staleness of up to twenty four hours, which is irrelevant for Indonesian Rupiah denominated used motorcycle prices that move on macro timescales.

Architectural commitment. For static cells the choice is precompute and cache. The system goes one step further at the demo layer, namely the precomputed cache is baked to per brand JSON files, shipped to the Vercel content delivery network, and queried from a TypeScript client. The deploy cost is zero on the Vercel hobby tier. The FastAPI service in `src/serving/api.py` is preserved as the production reference, and the architecture maps cleanly to Google Cloud Run or Vertex Endpoints in a paid deployment.

3.12 Mileage correction at serve time

The precomputed cell cache stores prices at a *reference mileage* per year of manufacture, namely $\bar{m}(y) = \max(1500, \text{age}(y) \cdot 8500)$ kilometers. At serve time the actual request carries an arbitrary mileage m . Rather than train a separate regressor on mileage as a continuous feature outside the LightGBM, the system applies a closed form piecewise linear correction on top of the cached cell value. Let $\bar{m}$ denote the reference mileage of the matched cell. Define the relative deviation

$$\Delta(m, \bar{m}) = \frac{m - \bar{m}}{\bar{m}}. \quad (13)$$

The correction factor is

$$\phi(m, \bar{m}) = \text{clip}(1 - 0.10 \cdot \Delta(m, \bar{m}), 0.78, 1.15), \quad (14)$$

and the served interval is the cached interval multiplied by ϕ on all three endpoints. The slope of -0.10 is empirically calibrated to the mileage feature importance in the trained LightGBM regressor. The clamp at $[0.78, 1.15]$ bounds the correction to the empirical range observed in the training distribution, so that adversarially low or high mileage inputs cannot drive the price arbitrarily far from the cached cell. Implementation lives in `src/serving/api.py:_mileage_adjustment` and

the mirrored TypeScript version in `web/lib/cache.ts:mileageAdjustment`. The two implementations carry the same constants by design, so the cached static demo and the live FastAPI service produce identical numbers on the same input.

3.13 Confidence inflation for cold start

When a request falls outside the training distribution, the system widens the predicted interval rather than refuse. Let $c = \frac{1}{2}(p_{\text{low}} + p_{\text{high}})$ denote the interval centre and $h = \frac{1}{2}(p_{\text{high}} - p_{\text{low}})$ denote the half width in log space. The inflated interval is

$$[c - \kappa \cdot h, c + \kappa \cdot h], \quad \kappa \in \{1.00, 1.15, 1.30\}, \quad (15)$$

where the multiplier κ depends on the confidence flag, namely $\kappa = 1.00$ for in distribution requests (flag `ok`), $\kappa = 1.15$ for an unseen kecamatan with a known brand model pair (flag `unseen_kecamatan`), and $\kappa = 1.30$ for an unseen brand model pair (flag `unseen_model`). The two values were chosen so that the unseen kecamatan widening is a smaller correction than the spatially weighted recalibration’s own per region fallback, and the unseen model widening is large enough to cover the empirical pricing range across the most recent Honda and Yamaha launches the training set did not yet include. Implementation lives in `src/models/predictor.py:_confidence`.

3.14 Deal tier classification

The CarGurus Instant Market Value widget displays each candidate price against a recommended value as a categorical label (Great Deal, Good Deal, Fair Deal, High Priced, Overpriced). The Indonesian language adaptation in the static demo uses three tiers, namely Di bawah rekomendasi, Sesuai rekomendasi, Di atas rekomendasi, with a symmetric tolerance band. Let p^* denote the recommended price and p_c denote the candidate price typed by the seller. The relative deviation is

$$\delta(p_c, p^*) = \frac{p_c - p^*}{p^*}. \quad (16)$$

The tier is

$$\tau(\delta) = \begin{cases} \text{below} & \text{if } \delta \leq -\theta, \\ \text{at} & \text{if } -\theta < \delta < \theta, \\ \text{above} & \text{if } \delta \geq \theta, \end{cases} \quad \theta = 0.04. \quad (17)$$

The tolerance θ at four percent is wider than typical observed price noise within a cell, so listings well inside the cell are classified as “Sesuai” rather than triggering false alarms on either side. Implementation lives in `web/lib/cache.ts:classifyDealTier`.

3.15 The labels question, listed price as a noisy proxy for sold price

The training label of the system is the listed price, because the marketplace observes listings but does not directly observe sold prices. Loberto, Luciani and Pangallo (2020), titled *What do online listings tell us about the housing market?*, study Italian housing classifieds and quantify the systematic gap between listed prices and realised sold prices. Their key empirical findings, namely listings reflect seller expectations and are biased upward relative to clearing prices, and the bias varies with regional liquidity, motivate the architectural commitment that the widget is *advisory* and *not binding*. The seller is free to deviate from the suggestion. The model is calibrated against listing time data because that is the only label the marketplace can observe at scale, but the architecture treats the listing price label as a noisy proxy rather than as ground truth.

3.16 Feedback loop discipline in advisory price suggestion

Tashman, Liang and Krishnan (2023), titled *Does Machine Learning Amplify Pricing Errors in the Housing Market?*, formalise the dynamic that arises when an advisory pricing model is widely adopted on a marketplace. Sellers anchor their listed price to the suggestion. The listing then becomes a future training label. The model retrains on labels it influenced. Under certain conditions the bias is reduced over iterations, and under other conditions the bias compounds. The paper gives sufficient conditions for convergence and divergence. The system inherits the following discipline from this result, namely the listing price training signal must be supplemented by a *permanent shadow cohort* of listings where the widget is turned off, so that the training distribution contains a sample of labels that were not influenced by the suggestion. The shadow cohort is the only causally clean training set under the advisory deployment. The v1 build does not yet maintain a shadow cohort, since the demo is not yet wired to a live marketplace, and this is on the v2 roadmap. The literature anchor is recorded in the theory so that the implementation is principled when added.

3.17 Marketplace experimentation, switchback design and interference bias

Once the widget is widely deployed, the natural question is how to A/B test improvements (a wider tolerance band, a new median model, a CV branch on photos). The textbook recipe of user level Bernoulli randomisation fails on a marketplace, because the treatment and control populations share the same inventory, and the cross pollination biases the estimated effect. Holtz, Lobel, Liskovich and Aral (2020), titled *Reducing Interference Bias in Online Marketplace Pricing Experiments*, study Airbnb pricing and measure interference bias on the order of thirty two percent of the estimated effect under user level randomisation. The right design is geographic cluster cross time switchback, in which kecamatan are clustered into balanced super cells and each super cell is randomised between A and B over weekly time blocks. Bojinov, Simchi-Levi and Zhao (2025), titled *Design and Analysis of Switchback Experiments*, give the canonical methodology paper. The v1 build does not include a switchback assignment service, because the demo runs in static mode without traffic, but the theory is recorded so that the production A/B framework, when added, follows a principled design rather than the default user level randomisation.

Why these three subsections are in the theory section. The labels question, the feedback loop discipline, and the switchback experimentation methodology are not yet directly implemented in the v1 code. They are recorded as theoretical foundations because they shape the architectural choices that *are* in the v1 code, namely the advisory-not-binding framing, the precompute-and-cache static demo (which makes the widget low risk to deploy without a feedback loop measurement system in place), and the deliberate absence of a user level A/B assignment service. The three citations were identified as load bearing in the original research synthesis at `D:\Research\Project Data\k1\research\used-vehicle-price-suggestion-2026-05-21\synthesis.md`.

4. Mathematical Summary

Every equation introduced above is collected in a single reference table, in the order it appears.

Eq.	Name and compact form	Role in Empu Harga
(1)	Conditional quantile $q_\alpha(x) = \inf\{y : \Pr(Y \leq y \mid X = x) \geq \alpha\}$	Three target quantiles at $\alpha \in \{0.1, 0.5, 0.9\}$. The median is the recommended price; the ten and ninety percent quantiles are the interval endpoints.
(2)	Pinball loss $L_\alpha(y, \hat{y}) = \max(\alpha(y - \hat{y}), (\alpha - 1)(y - \hat{y}))$	Objective for the three LightGBM regressors. Passed via <code>objective="quantile"</code> with the appropriate <code>alpha</code> .
(3)	CQR conformity score $E_i = \max(q_{0.1}(X_i) - Y_i, Y_i - q_{0.9}(X_i))$	Computed on the held out calibration partition by <code>CQRCalibrator.fit</code> .
(4)	CQR rank $r = \lceil (n + 1)(1 - \alpha) \rceil$	Position of the conformity score order statistic used as the calibration correction Q .
(5)	Conformalised interval $[q_{0.1}(x) - Q, q_{0.9}(x) + Q]$	The marginal CQR endpoints.
(6)	Coverage guarantee $\Pr(Y \in [q_{0.1}(X) - Q, q_{0.9}(X) + Q]) \geq 1 - \alpha$	Finite sample marginal coverage at level $1 - \alpha = 0.8$ under exchangeability.
(7)	Hierarchical kernel $w_i(x) \in \{1.00, 0.40, 0.10, 0.02\}$	Spatial proximity weight inside <code>SpatiallyWeightedCQR</code> .
(8)	Weighted quantile $Q(x) = \min\{t : \sum w_i \mathbf{1}[E_i \leq t] / \sum w_i \geq 1 - \alpha\}$	Per region SW CP correction.
(9)	Bayesian smoothing $\text{enc}(k) = (n_k \bar{y}_k + m \bar{y}_{p(k)}) / (n_k + m)$	Target encoding for kecamatan, kotamadya, and several composite categoricals; $m = 30$.
(10)	Population Stability Index $\sum_b (p_b - q_b) \log(p_b / q_b)$	Legacy drift score. Returns positive infinity on new categorical levels. Kept for comparison only.
(11)	Wasserstein 1 $W_1(P, Q) = \int F_P(x) - F_Q(x) dx$	The primary covariate drift score. Finite under support changes; reports in feature units.
(12)	Total variation $\frac{1}{2} \sum_a p_a - q_a $	Categorical analogue of W_1 . Used on brand, model, kecamatan distributions.
(13)	Mileage deviation $\Delta(m, \bar{m}) = (m - \bar{m}) / \bar{m}$	Relative deviation of the request mileage from the cell reference mileage. Input to the serve time correction.
(14)	Mileage correction factor $\phi = \text{clip}(1 - 0.10 \Delta, 0.78, 1.15)$	Piecewise linear clamp applied to the cached cell endpoints at serve time. Implementation in <code>src/serving/api.py</code> and <code>web/lib/cache.ts</code> .

Eq.	Name and compact form	Role in Empu Harga
(15)	Confidence inflation $[c - \kappa h, c + \kappa h], \kappa \in \{1.00, 1.15, 1.30\}$	Half width multiplier per confidence flag (ok, unseen kecamatan, unseen model). Cold start widening at the predictor layer.
(16)	Deal tier deviation $\delta(p_c, p^*) = (p_c - p^*)/p^*$	Relative deviation of the seller's candidate price from the recommended price.
(17)	Deal tier classifier $\tau = \text{below/at/above with threshold } \theta = 0.04$	CarGurus inspired three way classification of the candidate price against the recommendation.

5. Dataset

The modeling pipeline consumes an Indonesian used motorcycle listings dataset comprising approximately eighty thousand rows. The schema is as follows.

Schema. Each row represents one listing on an Indonesian used motorcycle marketplace and carries the following fields.

- `listing_id`: unique identifier of the listing.
- `brand`: motorcycle brand (Honda, Yamaha, Suzuki, Kawasaki, TVS, Vespa).
- `family`: brand family or product line (for example, Honda Vario, Yamaha NMAX).
- `model`: full model designation (for example, Vario 125, NMAX 155).
- `displacement_cc`: engine displacement in cubic centimetres.
- `body_type`: matic, sport, bebek, adventure.
- `drivetrain`: auto or manual.
- `msrp_2024_idr`: manufacturer suggested retail price at year of manufacture, in Indonesian Rupiah.
- `year`: year of manufacture.
- `age_years`: derived from year and the reference year (2026).
- `mileage_km`: odometer reading in kilometers.
- `kondisi`: condition tag in {mulus, normal, perlu perbaikan}.
- `kecamatan, kotamadya, provinsi`: administrative hierarchy of the listing location.
- `income_tier`: a one to five index for the kecamatan's median household income.
- `fair_price_idr, listed_price_idr`: fair value reference and the seller's listed price. `listed_price_idr` is the training label.
- `created_at, deactivated_at, is_deactivated`: lifecycle timestamps of the listing.

Coverage. The dataset spans Jabodetabek (Jakarta plus Bogor, Depok, Tangerang, Bekasi), seventy distinct kecamatan, thirty five distinct (brand, model) pairs, model years from 2015 through 2025, and three condition tags. Honda accounts for roughly sixty two percent of rows, Yamaha for thirty two percent, and the remaining six percent is distributed across Suzuki, Kawasaki, TVS, and Vespa. These proportions are consistent with the Mordor Intelligence 2025 Indonesian two wheeler market report.

Cross market reference set. A secondary dataset from the Indian two wheeler market (CarDekho on Kaggle) is loaded as a structural validation reference. The role of the secondary set is not training, but rather to verify that the feature importance ranking (mileage, age, brand, body type) is consistent across two distinct South and Southeast Asian markets, which supports the claim that the architecture generalises rather than being tied to one market’s idiosyncrasies. The cross market reference is loaded from the file `src/data/load_kaggle.py`.

6. End to End Data Flow

The system runs in two regimes, namely an offline training and calibration pipeline executed periodically (daily for the model, weekly for the calibration), and an online serving pipeline executed per request.

6.1 Offline pipeline

1. **Data loading.** The training dataset is loaded from `data/synthetic/listings_v1.parquet`. The reference taxonomy of brands and models is loaded from `src/synthesis/taxonomy.py` (a static dictionary of model specifications). The reference geography of Jabodetabek kecamatan is loaded from `src/synthesis/geography.py`.
2. **Feature engineering.** The script `src/features/build.py` consumes the listings and produces three Parquet files at `data/features/train.parquet`, `data/features/calibration.parquet`, `data/features/test.parquet`. The same script writes the fitted encoders to `data/features/encoders.pkl` and the column schema to `data/features/feature_schema.json`.
3. **Training.** The script `src/models/train_quantile.py` loads the three Parquet partitions, optionally tunes the median model via Optuna, trains the three quantile regressors, evaluates on the test partition, and writes the model bundle to `data/models/quantile_v1/quantile_bundle.pkl`. Metrics are logged to MLflow.
4. **Calibration.** The script `src/models/calibrate.py` loads the model bundle plus the calibration partition, fits the marginal CQR and the spatially weighted recalibration, evaluates coverage on the test partition, and writes the calibrated bundle to `data/models/quantile_v1/calibrated_bundle.pkl`. Coverage summary is saved to `data/models/quantile_v1/calibration_summary.json` and `data/models/quantile_v1/coverage_by_kecamatan.csv`.
5. **Stress test.** The script `src/models/stress_test.py` evaluates the calibrated bundle on three deliberately hard cohorts, namely rare brand, low volume kecamatan, and synthesised unseen model. Output is written to `data/models/quantile_v1/stress_test.json`.
6. **Cell precompute.** The script `src/serving/precompute.py` materialises the Cartesian product of the cell grid into a DuckDB file at `data/precompute/cells.duckdb`. The vectorised implementation builds the full feature matrix once and runs three batched LightGBM predictions, which is several orders of magnitude faster than per cell looping.

7. **Static bake.** The script `src/serving/prebake_to_json.py` reads the DuckDB cache, emits a manifest at `web/public/data/manifest.json`, and per brand compact JSON files at `web/public/data/honda.json`, `web/public/data/yamaha.json`, and so on.

6.2 Online pipeline

The static demo runs entirely in the browser. The flow is the following.

1. The user opens the page served from the Vercel content delivery network.
2. The TypeScript client at `web/lib/cache.ts` fetches `manifest.json` once.
3. When the user selects a brand, the client fetches the corresponding per brand JSON file lazily.
4. As the user fills in model, year, kecamatan, condition, the client computes the cell key, performs an $O(1)$ Map lookup over the loaded JSON, applies the mileage adjustment, and renders the three numbers in the `RangeBar` component at `web/components/RangeBar.tsx`.
5. The user can optionally type a candidate price into the input field. The component `web/components/DealTierBadge.tsx` classifies the candidate against the recommendation as “Di bawah rekomendasi”, “Sesuai rekomendasi”, or “Di atas rekomendasi”, with a four percent tolerance band on either side of the recommendation. The classifier is inspired by the CarGurus Instant Market Value deal tier visualisation.

If the demo is migrated to live inference, the orchestration moves to the FastAPI service at `src/serving/api.py`. The endpoints are `/health`, `/api/v1/models`, `/api/v1/kecamatan`, and `/api/v1/price-suggestion` (both GET and POST). The service first attempts an $O(\log N)$ lookup in the DuckDB cache, and on a cache miss falls back to live inference via the unified predictor at `src/models/predictor.py`.

7. Layer by Layer Architecture

The system is decomposed into independent layers so that each can be reasoned about and tested in isolation. The list below names every layer, lists the source files that implement it, and states the layer’s responsibility.

7.1 Layer 1, Data Reference and Loading

Files. `src/synthesis/taxonomy.py`, `src/synthesis/geography.py`, `src/data/load_kaggle.py`.

The taxonomy module defines thirty five Indonesian motorcycle models in a static lookup keyed by (brand, model), together with displacement, body type, drivetrain, and an MSRP value at year of manufacture. The geography module defines the seventy Jabodetabek kecamatan with their parent kotamadya and provinsi, and a one to five income tier index. The Kaggle loader fetches the secondary Indian two wheeler dataset for cross market validation and normalises it to a schema compatible with the primary dataset, with prices converted from Indian Rupees to Indonesian Rupiah at a fixed exchange rate.

7.2 Layer 2, Feature Engineering

Files. `src/features/encoding.py`, `src/features/split.py`, `src/features/build.py`.

The encoding module implements the hierarchical target encoder of Section 3.6. The split module implements the temporal split of Section 3.7. The build module orchestrates the full transformation. It produces thirty six features, namely twenty four numeric or one hot or target encoded columns and twelve categorical columns left for the LightGBM native handler. The

composite keys (brand colon family, brand colon displacement band, model colon year) are constructed from the raw columns and added as separate features. The temporal split is seventy percent training, fifteen percent calibration, fifteen percent test. The encoders are fitted on the training partition only and reused on calibration and test.

7.3 Layer 3, Model Training

Files. `src/models/metrics.py`, `src/models/train_quantile.py`.

The metrics module contains the pinball loss, the empirical coverage, the mean interval width, MAPE, and SMAPE as small NumPy functions. The training module fits three independent LightGBM regressors at $\alpha \in \{0.1, 0.5, 0.9\}$ with early stopping on the calibration partition. An optional Optuna run tunes the median model first, and the lower and upper regressors inherit the best hyperparameters. The script logs all metrics and the model bundle to MLflow with the experiment name “rekomendasi-harga”.

7.4 Layer 4, Calibration

Files. `src/models/conformal.py`, `src/models/calibrate.py`.

The conformal module implements the two calibrators. The marginal CQR is the class `CQRCalibrator`, with methods `fit` and `apply`. The spatially weighted version is the class `SpatiallyWeightedCQR`, with the hierarchical kernel of Equation (7) and the weighted quantile of Equation (8). A safety fallback returns the marginal correction when the weighted effective sample size drops below thirty. The calibrate script orchestrates the fit on the calibration partition, evaluates the three coverage levels (uncalibrated, marginal CQR, spatially weighted) on the test partition, prints a per volume decile table, and writes the calibrated bundle.

7.5 Layer 5, Unified Inference

File. `src/models/predictor.py`.

The class `PricePredictor` is the single entry point used by both the FastAPI service and the cell precompute script. The constructor loads the calibrated bundle and the fitted encoders. The method `predict_one` consumes a single request dictionary, builds the feature row, predicts the three log scale quantiles, applies the spatially weighted correction, applies the confidence inflation layer when the request is on an unseen brand model or an unseen kecamatan, exponentiates back to Indonesian Rupiah, and enforces the invariant `p_low` less than or equal to `p_recommended` less than or equal to `p_high`. The output is a `PriceQuote` dataclass with the three prices, the locale tag, the model version, the confidence flag, and a diagnostics dictionary.

7.6 Layer 6, Stress Test

File. `src/models/stress_test.py`.

The stress test exercises three cohorts.

- The rare brand cohort contains test rows whose brand is not Honda or Yamaha (the minor brands, about six percent of inventory).
- The low volume kecamatan cohort contains test rows whose kecamatan is in the bottom twenty percent by calibration volume.
- The unseen model cohort is the result of sampling two percent of the test set and renaming the model with a synthetic suffix (a placeholder for a freshly launched model that did not appear in training).

The script asserts that every cohort returns a valid interval and never raises. The output file `data/models/quantile_v1/stress_test.json` carries the cohort coverage, median width in Rupiah, median relative width, and the distribution of confidence flags. This file is referenced by the model card and the live demo Loom script.

7.7 Layer 7, FastAPI Service

File. `src/serving/api.py`.

The service exposes four endpoints, namely a health probe at `/health`, a model and brand listing at `/api/v1/models`, a kecamatan listing at `/api/v1/kecamatan`, and the price suggestion endpoint at `/api/v1/price-suggestion` (both GET and POST). The lifespan context opens the DuckDB cache read only and, optionally, the predictor for the live inference fallback. The endpoint logic is, first an $O(\log N)$ cache lookup keyed by (brand, model, year, kecamatan, kondisi), and second, if the cache lookup misses, a fall through to `PricePredictor.predict_one`. A linear mileage correction is applied on top of the cached cell, clamped to the range zero point seven eight to one point one five. The response schema is the same on cache hit and cache miss, with two extra fields `served_from` and `cache_hit` that disclose the path.

7.8 Layer 8, Cell Cache and Static Bake

Files. `src/serving/precompute.py`, `src/serving/prebake_to_json.py`.

The precompute script materialises the eighty thousand cell Cartesian product into a DuckDB read only file. The vectorised path builds the full feature matrix once and runs three batched LightGBM predictions. The full grid takes under two minutes on a laptop and produces an artifact under ten megabytes. The prebake script reads the same DuckDB file and emits a manifest of brands, models, and kecamatan together with one compact JSON file per brand. The compact encoding stores each cell as a fixed length integer array (model index, year, kecamatan index, condition index, reference mileage, three prices, confidence flag index), with strings deduplicated through the manifest. The resulting Honda file is one point seven megabytes; the Yamaha file is one point three megabytes; the small brand files are below half a megabyte each. Vercel automatically gzips static assets at the content delivery network edge, which reduces the over the wire payload by roughly four times.

7.9 Layer 9, Static Web Demo

Files. `web/app/layout.tsx`, `web/app/page.tsx`, `web/app/globals.css`, `web/components/PriceWidget.tsx`, `web/components/RangeBar.tsx`, `web/components/DealTierBadge.tsx`, `web/components/SelectField.tsx`, `web/components/NumberField.tsx`, `web/lib/cache.ts`, `web/lib/types.ts`, `web/lib/format.ts`.

The static demo is a Next.js sixteen application with TypeScript in strict mode and Tailwind CSS version four. The form orchestrator is `PriceWidget`. It fetches the manifest on mount and the per brand JSON lazily when the user selects a brand. As the user fills the fields, the client computes the cell key and performs an $O(1)$ lookup. The `RangeBar` component renders a horizontal track in marketplace blue with twenty one tick marks, a diamond marker at the recommended price, and an optional red ring at the user's typed candidate price. The `DealTierBadge` renders the green, blue, or red pill with the delta percentage. An "Expert mode" collapsible reveals the confidence flag, the reference mileage, the mileage adjustment factor, and the model version. The whole demo is served from the Vercel hobby tier at zero ongoing cost.

7.10 Layer 10, Drift Monitoring

File. `src/monitoring/drift.py`.

The drift module implements the Wasserstein 1 distance for numeric features, the total variation distance for categorical features, the legacy PSI for benchmarking, and an aggregating

function `compute_drift_report` that returns a Pandas DataFrame keyed by feature with columns `w1`, `psi`, `flag`. The alert threshold scales to the inter quartile range of the reference window for numeric features and to a fixed value for categorical features. A reproducible micro experiment in which the candidate window contains a new brand level returns positive infinity for PSI and a finite value for W1, which makes the case for the W1 substitution concrete.

7.11 Layer 11, Cost Modeling

File. `src/ops/cost_calc.py`.

The cost calculator computes the monthly Indonesian Rupiah cost of three deployment scenarios for a given queries per day load, namely the portfolio Vercel deploy (free), a production Google Cloud Platform native deploy, and a naive always on graphics processing unit endpoint. The constants at the top of the file are rates in United States Dollars for Cloud Composer, BigQuery, Vertex AI Training, Cloud Run, Memorystore Redis, and Cloud Logging. The output table prints monthly cost in both United States Dollars and Indonesian Rupiah, and the per thousand suggestions unit cost. The production versus naive graphics processing unit ratio is reported, and quantifies the value of the central processing unit only serving choice.

7.12 Layer 12, Preflight Verification

File. `tools/preflight.py`.

The preflight script runs eight independent checks before any deploy. The checks are, abstract syntax tree parsing of every Python file under `src/` and `tests/`, imports of every `src` module, an end to end mini pipeline (data preparation, feature engineering, prebake to JSON), the drift detector with the W1 versus PSI comparison, the conformal calibrators including the unseen region fallback, the cost calculator with the production over naive ratio invariant, a guarded predictor module import, and a FastAPI cache hit smoke test backed by a temporary DuckDB mock. The whole sequence executes in under two seconds on a laptop.

8. Story, Mistakes and Corrections

The system did not emerge clean. The build was a fourteen day sprint with several mid course corrections that are worth recording, because they illustrate the kind of judgement that distinguishes a working portfolio from a brittle one.

The Streamlit to Next.js swap. The original build plan called for a Streamlit demo, because Streamlit is the path of least resistance for data engineers. After Day Eight the demo was running on Streamlit, but the visual quality was inadequate for a portfolio piece intended to be shown to multiple companies. Streamlit's default boxy components do not communicate engineering taste. The decision was taken to throw away the Streamlit demo and rebuild on Next.js sixteen with Tailwind version four. The replacement provides a pixel level replica of the live marketplace widget, runs on the Vercel content delivery network at zero ongoing cost, and demonstrates breadth from machine learning into product and deployment. The Streamlit code was deleted, the `Dockerfile.ui` replaced by `Dockerfile.web`, and the continuous integration workflow split into two jobs, namely Python and Web.

The per cell precompute disaster. The first version of `src/serving/precompute.py` looped `PricePredictor.predict_one` over the entire eighty thousand cell grid. Each call rebuilt the full feature vector for a single row, ran three single row LightGBM predictions, and applied the spatially weighted correction. On a laptop the loop projected to fourteen hours and saturated the central processing unit at very low throughput. The diagnosis was straightforward, namely the per row prediction was paying the LightGBM batch start up cost eighty thousand times.

The fix was to rewrite the scoring function as a vectorised pipeline that builds the full feature matrix once, runs the three LightGBM predictions on the whole matrix, applies the spatially weighted correction in a loop that calls only the lightweight `offset_for` method per cell, and clamps the output. The vectorised path completes in ninety seven seconds for eighty thousand cells, namely about eight hundred and thirty cells per second.

The Next.js security advisory. The first Vercel deploy used Next.js sixteen point zero point zero, which was the latest stable version at scaffold time. Vercel itself refused the deploy with an automated security advisory and a request to upgrade. The fix was to bump to Next.js sixteen point two point six and React nineteen point two, which is the latest stable line at the time of writing. The lesson is, even on a static deploy, the dependency surface still requires routine maintenance.

The disk space failure. The system Python on the laptop sits on the C drive, which had only zero point two gigabytes free at the start of the build. The first attempt at `pip install` failed mid stream with an out of space error. The fix was to set the pip temporary directory and the pip cache directory to the D drive (fifteen gigabytes free) via environment variables, after which the install completed cleanly. The lesson is, observe the environment before assuming it.

The Pandas three dtype gotcha. The feature builder uses an internal function `_is_categorical` to decide which columns to leave as native LightGBM categorical and which to encode. The initial implementation tested only for the dtypes `object` and `category`. Pandas version three (released between this build and the previous one) introduced a new default string dtype, namely the string `str`. The initial test mislabelled the new dtype as numeric, and the LightGBM training raised. The fix was a one line patch that added `str` and `string` prefixes to the check.

The marketplace agnostic rewrite. The original documentation, including the README and the inline copy on the demo page, was tied to a specific Indonesian marketplace company by name. The portfolio is intended to be submitted to multiple companies. The decision was taken to sweep the repository and replace every company specific reference with a marketplace agnostic framing, namely “Indonesian used motorcycle marketplace”, “the marketplace widget”, and so on. Twenty four files were touched. The build verifier was rerun and continued to pass eight out of eight checks. The git history was squashed to a single clean commit and force pushed to the remote.

9. Results

The numbers below are produced by running the offline pipeline on the working dataset with twenty Optuna trials on the median LightGBM model.

Hyperparameter search. Optuna picked one thousand one hundred and seventy three estimators, a learning rate of zero point zero two two, seventy seven leaves, a minimum child sample count of eighty, a feature fraction of zero point six seven, a bagging fraction of zero point seven three, and weak L1 and L2 regularisation. The three quantile regressors share the same configuration and stop on the calibration partition at iteration eight hundred and two for the lower model, six hundred and fifty six for the median model, and eight hundred and thirty eight for the upper model.

Point accuracy at the median. Median absolute percentage error on the test partition is four point three four percent. Mean absolute percentage error on the calibration partition is four

point two eight percent. Pinball loss at the median is zero point zero two two on test and zero point zero two one on calibration. The values are stable across partitions, which indicates no overfitting.

Interval coverage. Coverage at the target level of eighty percent is reported in three regimes, namely uncalibrated (the raw LightGBM quantile output), marginal CQR, and spatially weighted CP. The uncalibrated coverage is seventy three point seven percent, which is the classic vanilla quantile undercoverage. Marginal CQR coverage is seventy nine point seven seven percent. Spatially weighted CP coverage is seventy nine point seven nine percent. Both calibrated coverages are well within tolerance of the target. The complete coverage summary is saved to `data/models/quantile_v1/calibration_summary.json`.

Per volume decile coverage. A nuanced view of coverage is the per decile breakdown, where the deciles are formed by kecamatan calibration volume (from sparsest to densest). The uncalibrated coverage ranges from zero point seven one six to zero point seven six four across deciles, the marginal CQR coverage ranges from zero point seven seven four to zero point eight two three, and the spatially weighted CP coverage ranges from zero point seven seven nine to zero point eight two two. The spatially weighted version delivers the tightest spread, which is the empirical signature the calibration layer is designed to deliver. The per kecamatan coverage table is saved to `data/models/quantile_v1/coverage_by_kecamatan.csv`.

Interval width. The mean width of the spatially weighted interval is about fourteen point two percent relative to the mean price. Interpreted at a representative cell, for a recommended price of fifteen point seven million Indonesian Rupiah, the interval is approximately fourteen point nine to sixteen point five million.

Stress test. Output is saved to `data/models/quantile_v1/stress_test.json`. The full test cohort returns coverage of seventy nine point eight percent at a median relative width of thirteen point nine percent. The rare brand cohort returns coverage of seventy nine point six percent at a median relative width of fourteen point three percent. The low volume kecamatan cohort returns coverage of seventy eight point six percent at a median relative width of thirteen point eight percent. The synthesised unseen model cohort returns coverage of twenty percent at a wider median relative width of nineteen point nine percent and surfaces the confidence flag “unseen_model” on all rows. The widget can therefore render an honest “estimasi luas” badge rather than refuse, and the lower coverage on the unseen model cohort is the expected behaviour, namely a model that did not appear in training cannot be priced with confidence.

Drift detector. On a synthetic micro experiment in which the candidate window introduces two new brand levels (Kawasaki and Vespa) on top of the reference window’s Honda, Yamaha, and Suzuki, the legacy PSI returns positive infinity on the brand column. The Wasserstein 1 based total variation on the same column returns zero point one seven seven. The contrast confirms the architectural choice and provides a concrete demonstration that runs in under one second on a laptop.

Cost. For a representative production load of one million suggestions per day, the cost calculator prints three scenarios. The portfolio Vercel deployment is zero Indonesian Rupiah per month. The production Google Cloud Platform native deployment is approximately five point six million Indonesian Rupiah per month, namely about one hundred and eighty eight Indonesian Rupiah per one thousand suggestions. A naive always on graphics processing unit baseline is approximately twenty seven million Indonesian Rupiah per month, which is roughly four point eight times the central processing unit baseline at no accuracy benefit.

Preflight. Every check in `tools/preflight.py` passes on the working build. The full sequence runs in under two seconds.

Live demo. The Vercel deployment serves the static HyperText Markup Language with no cold start. A request for Honda Vario 125 manufactured in 2018, sixty thousand kilometers, normal kondisi, Cilandak (a kecamatan in Jakarta Selatan) returns three numbers approximately equal to fourteen point nine million, fifteen point seven million, and sixteen point five million Indonesian Rupiah, with the locale tag “Berlaku untuk Cilandak, Jakarta Selatan”.

Reading the results. The marginal coverage at the target level is met with two percentage points of tolerance. The conditional coverage across volume deciles is held within a four percentage point band by the spatially weighted layer. The point accuracy at the median is below five percent absolute percentage error. The drift detector demonstrates the practical benefit of Wasserstein 1 over the legacy Population Stability Index. The cost calculator quantifies the savings of the precompute and cache architecture. Every output above is reproducible from the codebase by running the offline pipeline in order.

10. Limitations

The system is honest about what it does not do.

- **Absolute Indonesian Rupiah prices are illustrative.** The recommended price for a specific cell is calibrated against public manufacturer suggested retail prices and standard depreciation curves. The methodology generalises to live marketplace data unchanged, but absolute values are not benchmarked against real marketplace listings.
- **No image features.** The condition tag is the only condition signal in the v1 system. A computer vision branch on listing photographs is the most obvious lever for accuracy improvement (Section 11).
- **No description text.** A free text description field is not used. A modest accuracy lift is expected from an Indonesian language transformer on the description text, namely IndoBERT or a similar encoder.
- **No electric motorcycle category.** The electric two wheeler segment is approximately one percent of the 2024 Indonesian two wheeler market. The training data does not contain enough electric listings to model the category separately. The system falls back to the dominant internal combustion engine category.
- **Geographic scope is Jabodetabek only.** The seventy kecamatan included are in the Jakarta metropolitan region. Out of metropolitan requests fall back to the provinsi or national mean, which is honest behaviour but reduces the value of the recommendation outside the trained scope.
- **Two wheelers only.** The product is restricted to motorcycles. Used cars (four wheel) require additional condition fields (interior, exterior, accident history) and a different reference taxonomy.
- **No A/B experimentation backend.** The widget can be A/B tested by switching the displayed numbers between model versions, but the experiment requires a server side assignment service that is out of scope for the static demo. The right experimentation pattern is geo cluster cross time switchback (Bojinov, Simchi-Levi and Zhao, 2025) to avoid marketplace interference bias (Holtz, Lobel, Liskovich and Aral, 2020).

- **Anchor effect not measured.** If the widget is widely adopted, sellers will anchor their listed price to the recommendation. The recommendation then influences future training labels through the price they encourage sellers to list. The feedback loop discipline of Tashman, Liang and Krishnan (2023) is documented in the design and is a v2 implementation item, namely a permanent shadow holdout cohort where the widget is off so that the listed price distribution remains causally clean for retraining.
- **Model drift detection is not wired to an alerting system.** The drift module is operational and can be invoked from a Python session, but a production system would wire it into a Slack or email webhook with a scheduled job. This is on the v2 roadmap.

11. Potential Future Projects

The architecture extends cleanly along several axes.

Image condition branch. Add a small convolutional neural network that consumes listing photographs and produces a condition embedding of size eight to sixteen. The embedding joins the LightGBM feature set as additional columns. The expected accuracy lift is on the order of three to five percent absolute percentage error at the median, based on the Carro one hundred and twenty point inspection precedent and similar published cases. The compute moves to a graphics processing unit for the offline embedding pass only; the serving path remains central processing unit because the embedding is a precomputed feature.

Indonesian language description encoder. Fine tune an Indonesian language transformer on listing descriptions and use the resulting embedding as a feature. The relevant base models are IndoBERT (Wilie et al., 2020) and the family of multilingual encoders that include Indonesian, namely XLM RoBERTa.

Kecamatan graph embedding. Replace the target encoder for kecamatan with a node embedding learned on a graph whose nodes are kecamatan and whose edges encode physical adjacency plus functional similarity (commuter flows, income brackets, BPS demographic vectors). The pattern is well established for real estate automated valuation by Peng, Liu, Wang and Jin (2020), titled *Lifelong Property Price Prediction*, and would specifically help cold start in low volume kecamatan.

Geo cluster cross time switchback. Build a server side assignment service that randomises kecamatan clusters into A and B over time blocks. Reference designs are the switchback methodology of Bojinov, Simchi-Levi and Zhao (2025), and the production case at LATAM Airlines (Ferrari-Ortiz et al., 2026). The experiment design avoids the user level Bernoulli randomisation bias measured at thirty two percent on Airbnb pricing experiments by Holtz, Lobel, Liskovich and Aral (2020).

Feedback loop measurement. Reserve a permanent shadow cohort where the widget is turned off, indefinitely. Retrain only on the shadow cohort data so that the training set remains causally clean. Track the divergence between widget enabled and widget disabled distributions as the metric of the anchor effect. The framework is Tashman, Liang and Krishnan (2023).

Multi market generalisation. Extend to Greater Surabaya, Greater Bandung, and Medan by training a regional model per metropolitan area. The architecture is unchanged. The taxonomy and the geography modules need a Surabaya, Bandung and Medan replacement.

Edge inference via ONNX. Port the trained LightGBM regressors to the Open Neural Network Exchange format and run inference inside a Vercel Edge Function. The advantage is the removal of the static cell grid limitation, namely the system can serve live for combinations that the precompute did not cover.

Used car extension. Move from two wheelers to four wheelers. The product schema requires additional fields, namely interior condition, exterior condition, service history, accident history, and the brand model taxonomy needs to be replaced. The calibration framework (CQR plus SW CP) remains unchanged.

12. Repository Layout

The full set of files is listed below for orientation. The reader is expected to navigate to a file by name when the document references it.

README.md	top level frontdoor (problem, results, deploy)
PROJECT_PLAN.md	day by day sprint log
pyproject.toml	Python project metadata and dependencies
requirements.txt	flat dependency list for Docker
Makefile	one command targets (install, data, train, ...)
Dockerfile.api	FastAPI service image
Dockerfile.web	Next.js production image
docker-compose.yml	local two service composition
.github/workflows/ci.yml	Python and Web continuous integration
src/data/load_kaggle.py	cross market reference loader
src/synthesis/taxonomy.py	motorcycle reference taxonomy
src/synthesis/geography.py	Jabodetabek kecamatan reference
src/synthesis/generate.py	dataset preparation script
src/features/encoding.py	hierarchical target encoder
src/features/split.py	temporal split utility
src/features/build.py	feature pipeline orchestrator
src/models/metrics.py	pinball, coverage, MAPE, SMAPE
src/models/conformal.py	CQR and SW CP calibrators
src/models/train_quantile.py	LightGBM quantile training with Optuna and MLflow
src/models/calibrate.py	calibration orchestrator
src/models/predictor.py	unified inference interface
src/models/stress_test.py	cold start cohort tests
src/serving/precompute.py	cell cache materialiser
src/serving/api.py	FastAPI service
src/serving/prebake_to_json.py	JSON bake for the static demo
src/monitoring/drift.py	Wasserstein 1 and PSI drift report
src/ops/cost_calc.py	monthly cost calculator
tools/preflight.py	eight check preflight verifier
tests/	nine test files, fifty three test functions
web/app/	Next.js App Router pages
web/components/	TypeScript React components
web/lib/	TypeScript client logic (cache, types, format)
web/public/data/	manifest plus per brand prebaked JSON
docs/	this document plus the supplementary docs

13. Disclaimer

Empu Harga is a portfolio demonstration of methodology. It is not financial advice and is not a commercial pricing service. Absolute Indonesian Rupiah values are illustrative. Real marketplace deployment requires a current listings dataset, periodic retraining, and an alerting pipeline on the drift detector.

A. Annotated Bibliography

Every source below shaped a concrete decision in Empu Harga. For each, we state the citation, the headline claim, and how Empu Harga operationalised it. Entries are grouped by the theoretical section that uses them.

A.1 Quantile regression and conformal calibration

Koenker, R. and Bassett, G. (1978). *Regression Quantiles*. *Econometrica*, 46(1):33–50.

The original paper on quantile regression with the asymmetric loss function (later called the pinball loss). Establishes the loss minimiser as the conditional quantile. **Used by Empu Harga:** Section 3.3 (loss derivation); Section 7.3 (`src/models/train_quantile.py` sets `objective="quantile"`).

Romano, Y., Patterson, E. and Candès, E. (2019). *Conformalized Quantile Regression*. *NeurIPS 2019*. [arXiv:1905.03222](#).

Wraps any quantile regressor with a finite sample marginal coverage guarantee under exchangeability. Defines the conformity score as the maximum of the lower miss and the upper miss. The order statistic of the calibration scores becomes the symmetric correction added to both endpoints. **Used by Empu Harga:** Section 3.4; Section 7.4 (`src/models/conformal.py:CQRCalibrator`).

Hjort, J., Hermansen, P., Pensar, J. and Williams, A. (2025). *Uncertainty Quantification in Automated Valuation Models with Spatially Weighted Conformal Prediction*. [arXiv:2312.06531](#).

Reweights calibration conformity scores by spatial proximity to the test point, recovering conditional coverage in low volume regions where marginal CQR systematically over or under covers. The original formulation uses a Gaussian kernel on centroid coordinates. Empu Harga adopts a discretised hierarchical kernel because Indonesian administrative geography is categorical. **Used by Empu Harga:** Section 3.5; Section 7.4 (`src/models/conformal.py:SpatiallyWeightedCQR`).

A.2 Gradient boosting on tabular data

Grinsztajn, L., Oyallon, E. and Varoquaux, G. (2022). *Why do tree based models still outperform deep learning on tabular data?*. *NeurIPS 2022 Datasets and Benchmarks*. [arXiv:2207.08815](#).

Systematic benchmark across forty five tabular datasets. Tree based ensembles dominate transformer and deep tabular baselines at the ten thousand to one million row scale. **Used by Empu Harga:** Section 3.2; underwrites the choice of LightGBM over a deep tabular model.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. *NeurIPS 2017*.

Original LightGBM paper. Introduces leaf wise growth, gradient based one side sampling, exclusive feature bundling. **Used by Empu Harga:** Section 7.3 (the trained model is a LightGBM ensemble).

Madhusudhanan, K., Hartwig, J., Loni, M. and Schmidt-Thieme, L. (2024). *ProbSAINT: Probabilistic Tabular Regression for Used Car Pricing*. [arXiv:2403.03812](#).

Self attention based deep tabular model that lands probabilistic calibration on used car prices. **Used by Empu Harga:** Section 3.2; considered and deferred to a v1.5 research ablation.

A.3 Hyperparameter search

Bergstra, J., Bardenet, R., Bengio, Y. and Kégl, B. (2011). *Algorithms for Hyperparameter Optimization*. NeurIPS 2011.

The Tree structured Parzen Estimator. Two densities, ℓ for good configurations and g for the rest. Maximise $\ell(x)/g(x)$. **Used by Empu Harga:** Section 3.10; Section 7.3 (the Optuna sampler used to tune the median model).

Akiba, T., Sano, S., Yanase, T., Ohta, T. and Koyama, M. (2019). *Optuna: A Next generation Hyperparameter Optimization Framework*. KDD 2019.

Software framework that wraps the TPE sampler and provides early stopping, pruning, and persistent storage. **Used by Empu Harga:** Section 7.3 (the `-tune -n-trials 20` mode invokes the Optuna study).

A.4 Drift detection and monitoring

Karras, J., Yip, C., Page, S. and Conway, C. (2002). *Population Stability Index in Credit Scoring*. Credit Risk International.

The original PSI formulation, widely adopted in finance. Returns the Kullback Leibler style divergence on histogram bins. Breaks (returns infinity) when a bin has zero mass in either window. **Used by Empu Harga:** Section 3.9; Section 7.10 (`src/monitoring/drift.py` keeps PSI for benchmark only).

Villani, C. (2009). *Optimal Transport: Old and New*. Springer.

The textbook reference for the Wasserstein distance and the broader theory of optimal transport. **Used by Empu Harga:** Section 3.9; Section 7.10 (`src/monitoring/drift.py` implements the one dimensional Wasserstein 1 via the cumulative distribution function integral).

Ramdas, A., Garcia-Trillos, N. and Cuturi, M. (2017). *On Wasserstein Two Sample Testing and Related Families of Nonparametric Tests*. Entropy, 19(2):47.

Establishes the finite sample sum form of one dimensional Wasserstein 1 computable in linearithmic time. **Used by Empu Harga:** Section 3.9.

A.5 Marketplace pricing references

Tashman, A., Liang, R. and Krishnan, S. (2023). *Does Machine Learning Amplify Pricing Errors in the Housing Market?*. arXiv:2302.09438.

Formal analysis of the feedback loop in which an advisory pricing model influences seller listing decisions, which then become training labels. Proves conditions under which the loop converges versus diverges. **Used by Empu Harga:** Section 3.16 (foundational discipline motivating the permanent shadow holdout cohort on the v2 roadmap).

Holtz, D., Lobel, R., Liskovich, I. and Aral, S. (2020). *Reducing Interference Bias in Online Marketplace Pricing Experiments*. arXiv:2004.12489.

Empirical study on Airbnb measuring marketplace interference bias under user level randomisation. The user level randomisation bias is on the order of thirty two percent of the estimated effect. **Used by Empu Harga:** Section 3.17 (the empirical anchor that motivates rejecting user level randomisation).

Bojinov, I., Simchi-Levi, D. and Zhao, J. (2025). *Design and Analysis of Switchback Experiments*. arXiv:2009.00148.

The canonical methodology paper on switchback experiments in marketplaces. **Used by Empu Harga:** Section 3.17 (the chosen A/B design for any production roll out of the widget).

Loberto, M., Luciani, A. and Pangallo, M. (2020). *What do online listings tell us about the housing market?.* arXiv:2004.02706.

Quantifies the systematic gap between listed prices and sold prices on Italian housing listings. Establishes the structural label problem on classifieds platforms. **Used by Empu Harga:** Section 3.15 (the foundational justification for the advisory-not-binding architectural framing).

Peng, X., Liu, X., Wang, S. and Jin, L. (2020). *Lifelong Property Price Prediction: A Case Study on Real Estate Markets with Graph Neural Networks.* arXiv:2008.05880.

Graph neural network on a graph of properties for real estate automated valuation. The closest published precedent for the future kecamatan graph extension. **Used by Empu Harga:** Section 11.

A.6 Industry case studies

Zillow Tech (2021). *Building the Neural Zestimate.*

Engineering blog. Convolutional plus fully connected architecture. Median error rate of six point nine percent on approximately one hundred and four million United States dwellings. Daily updates. **Used by Empu Harga:** Section 1 (the canonical iBuying versus advisory contrast); the Zillow Offers wind down is the cautionary capital exposure tale.

CarGurus (2024). *What is Instant Market Value?.* Help center documentation.

CarGurus Instant Market Value uses a comparable listings methodology with mileage and trim adjustments and updates daily across more than one million United States listings. Deal tier badges (Great Deal, Good Deal, Fair Deal, High Priced, Overpriced) are part of the public display. **Used by Empu Harga:** Section 3.14 (the deal tier classifier formal definition) and Section 7.9 (the DealTierBadge component).

Airbnb Engineering (2022). *Learning Market Dynamics for Optimal Pricing.*

Hybrid machine learning and structural model for host facing smart pricing. Parametric Gamma distribution on lead time densities. **Used by Empu Harga:** Section 1 (the closest advisory pricing analogue outside vehicles).

Wayfair Engineering (2022). *How Wayfair Uses Geo Experiments to Measure Incrementality.*

Production case study of geo cluster experiments in an e commerce marketplace. **Used by Empu Harga:** Section 11 (precedent that geo cluster switchback generalises beyond rideshare).

AIM Group (2025). *Avito Auto leverages AI to evaluate used cars for sale.*

Industry coverage of the November 2025 launch of the Avito Auto Assessment in Russia, which is the closest functional analogue to the Indonesian widget. **Used by Empu Harga:** Section 1 (the closest peer launch).

A.7 Indonesian market and geography references

Mordor Intelligence (2025). *Indonesia Two Wheeler Market Report 2025–2030.*

Market sizing of the Indonesian two wheeler segment, brand share, internal combustion versus electric mix. **Used by Empu Harga:** Section 1 (the Honda plus Yamaha share of ninety seven percent that motivates the intra Honda taxonomy emphasis).

MotorcyclesData (2025). *Indonesia Motorcycles: Data and Facts 2025.*

Monthly sales data for the Indonesian two wheeler market by brand and model. **Used by Empu Harga:** Section 1.

International Council on Clean Transportation (2025). *Electric Vehicle Market in Indonesia.*

Penetration of electric two wheelers in the Indonesian market. Approximately one percent of 2024 sales. **Used by Empu Harga:** Section 10 (justifies the electric segment being out of scope for v1).

Badan Pusat Statistik (BPS). *Administrative classification, Jabodetabek.*

Standard administrative classification of provinsi, kotamadya, kabupaten, kecamatan, and the income tier statistics used as a feature in the model. **Used by Empu Harga:** Section 7.1 (src/synthesis/geography.py).

A.8 Tabular methods, additional

Chen, T. and Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System.* KDD 2016.

The companion implementation of gradient boosted trees. Empu Harga does not use XGBoost directly; the reference is included for orientation only. **Used by Empu Harga:** orientation reading.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. and Gulin, A. (2018). *CatBoost: unbiased boosting with categorical features.* NeurIPS 2018.

Ordered target encoding to avoid target leakage. **Used by Empu Harga:** Section 3.6 (alternative considered; LightGBM with explicit Bayesian smoothing was chosen for transparency).

Wilie, B., Vincentio, K., Winata, G., Cahyawijaya, S., Li, X., Lim, Z., Soleman, S., Mahendra, R., Fung, P., Bahar, S. and Purwarianti, A. (2020). *IndoNLU: Benchmark and Resources for Evaluating Indonesian Natural Language Understanding.* ACL 2020.

Introduces IndoBERT and the Indonesian language understanding benchmark. **Used by Empu Harga:** Section 11 (the candidate description text encoder if a v2 NLP branch is added).

A.9 Software references

Pedregosa, F. et al. (2011). *scikit learn: Machine Learning in Python.* JMLR 12.
scikit learn is used for the train calibration test split utilities (the temporal split is implemented directly in src/features/split.py) and the metrics used in evaluation. **Used by Empu Harga:** Section 7.2.

Harris, C. et al. (2020). *Array Programming with NumPy.* Nature, 585.

NumPy underpins every numerical operation. **Used by Empu Harga:** throughout.

McKinney, W. (2010). *Data Structures for Statistical Computing in Python.* SciPy 2010.

Pandas is the tabular layer of the project. **Used by Empu Harga:** throughout.

Raasveldt, M. and Mühleisen, H. (2019). *DuckDB: an Embeddable Analytical Database.* SIGMOD Demonstration 2019.

DuckDB is the embedded analytical engine that stores the precomputed cell cache. **Used by Empu Harga:** Section 7.8.

Ramachandran, S. (2019). *FastAPI documentation.*

FastAPI is the asynchronous Python web framework that exposes the production reference service. **Used by Empu Harga:** Section 7.7.

Vercel Inc. (2026). *Next.js 16 documentation*.

Next.js with the App Router and the Turbopack bundler hosts the static demo. **Used by Empu Harga:** Section [7.9](#).

Tailwind Labs (2026). *Tailwind CSS v4 documentation*.

Tailwind v4 provides the utility class styling and the colour token system. **Used by Empu Harga:** Section [7.9](#) (`web/app/globals.css`).

Anonymous (2024). *kagglehub documentation*.

The kagglehub library provides programmatic access to Kaggle datasets, used to load the Indian two wheeler dataset for cross market validation. **Used by Empu Harga:** Section [7.1](#) (`src/data/load_kaggle.py`).